Team Member Full Name	NetID
Matthew Lee	**mlee55**
Sofia Cardona	**scardon2**
Martin Castellanos	**mcastel5**

Features Implemented for Phase 1

- Feature 1.1: Create new User Profile
- Feature 2: Gameplay
- Feature 3.1: Dashboard view of all prior plays

Persistent Storage Design

Describe here (in details) how your project is persisting data. Describe its format and schema. Include diagrams (all figures should have captions).
For example: "We are using SQLite database to persist our data. Our database includes the tables shown in Figure 1. It contains N tables. The `Question` table…"

We are using SQLite to persist data for Phase 1. For subfeature 1.1, our storage design uses Django's built-in User model together with a custom PlayerProfile model as shown in Figure 1.

The User model stores the required account information for profile creation, including username, email, and password. We chose this design because Django already provides a secure and standard way to manage authentication data, including password hashing and account handling. This avoids reimplementing authentication logic manually.

The PlayerProfile model stores application-specific player information, which currently includes the optional name field. Each PlayerProfile is connected to exactly one User through a one-to-one relationship. Separating authentication data from player-specific data keeps the design modular, easier to maintain, and easier to extend in later phases if additional profile fields are added.

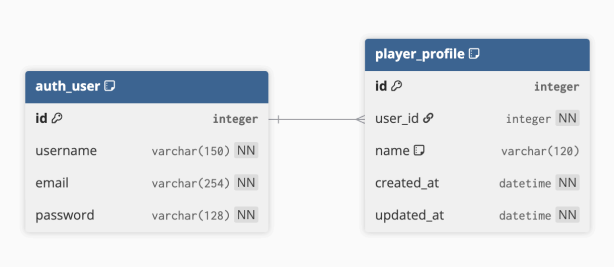

Figure 1 database schema subfeature 1.1

Demonstration of the Features Implemented for Phase 1

It should have at least one screenshot per feature. This should include not only the screenshots but also an explanation for them! Make sure the screenshots have good resolution (i.e., they can be readable when printed).

Feature 1.1: Create new User Profile

Subfeature 1.1 allows a user to create a new player profile in Worndly. On the profile creation page, the user enters an optional name and the required username, email, and password fields. When the form is submitted, the system validates the input and creates a new account.

After successful profile creation, the system stores the user's information in the database and redirects the user to their profile page, where the created profile information can be viewed. This feature establishes the initial account/profile creation flow for the application.

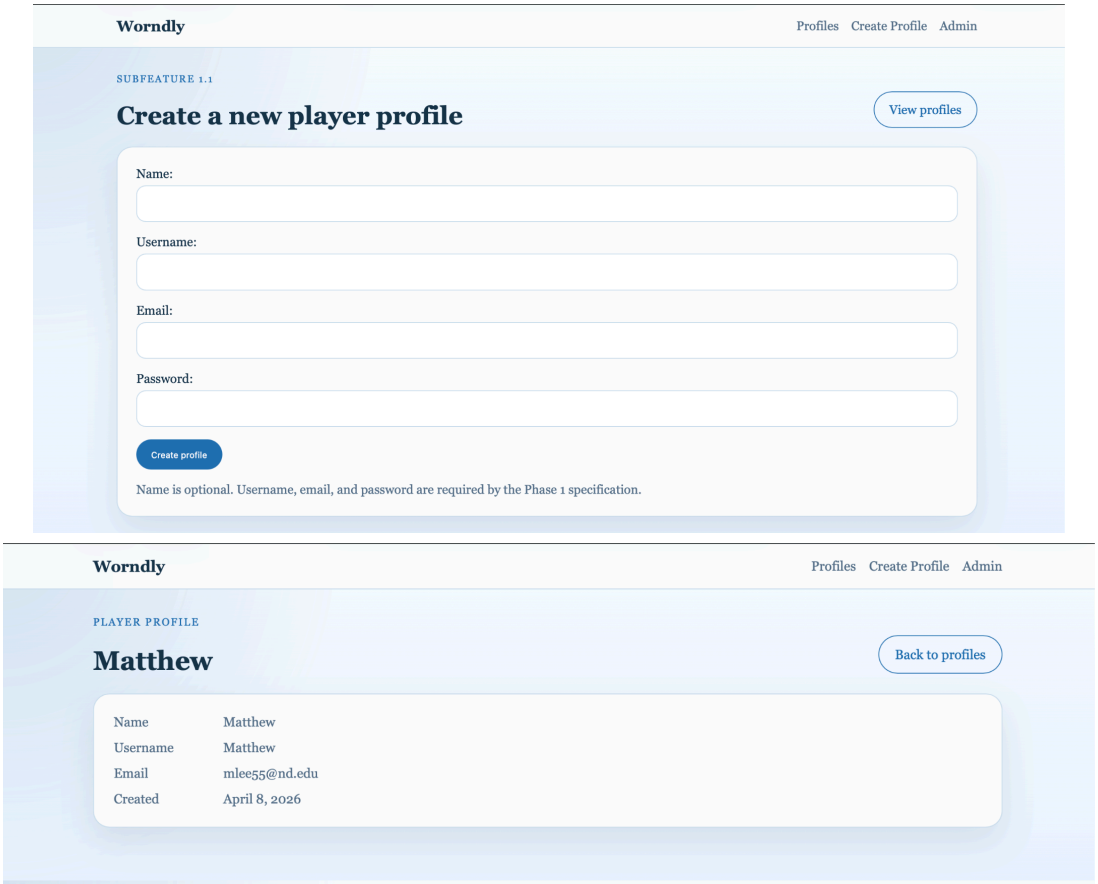

Figure 2 Screenshot for feature 1.1 showing creating player profile and created player profile

Feature 2: Gameplay

This feature allows logged users to play WorNDly, which is a Wordle-like game. The user can select one of four languages (english, spanish, french, german, portuguese) and ‘Start Game’. The users will be presented with blank tiles to fill in with five-letter words and have six attempts. In addition, the user is allowed three games per day. As the player inputs words, each letter is marked with a representative color: gray – letter not in word, yellow – letter in word but in wrong spot, green – correct letter in corresponding spot. If they input an invalid word, an error appears. Once the user guesses the correct word they are given the choice to play again; if they get it incorrect, the correct word is displayed and are given the choice to play again.

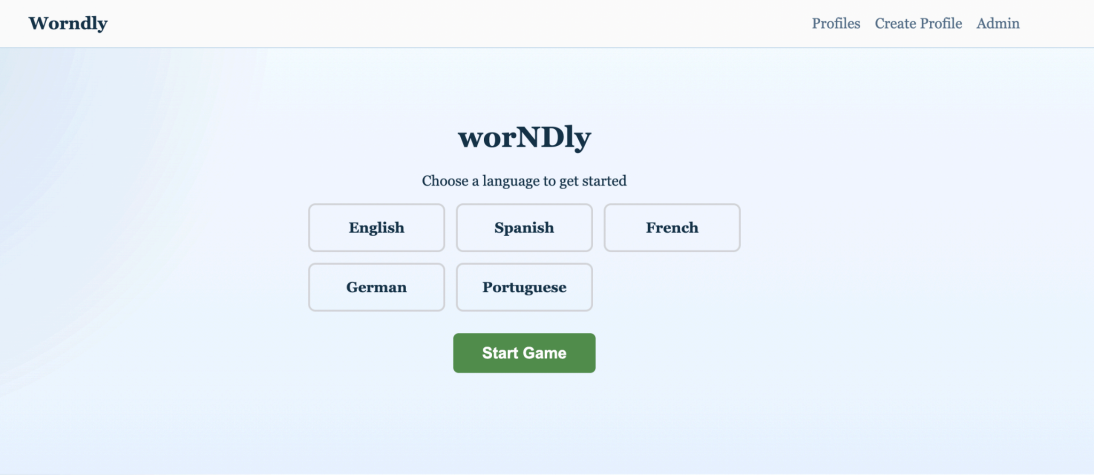

Figure 1. Screenshot of feature 2 language selection

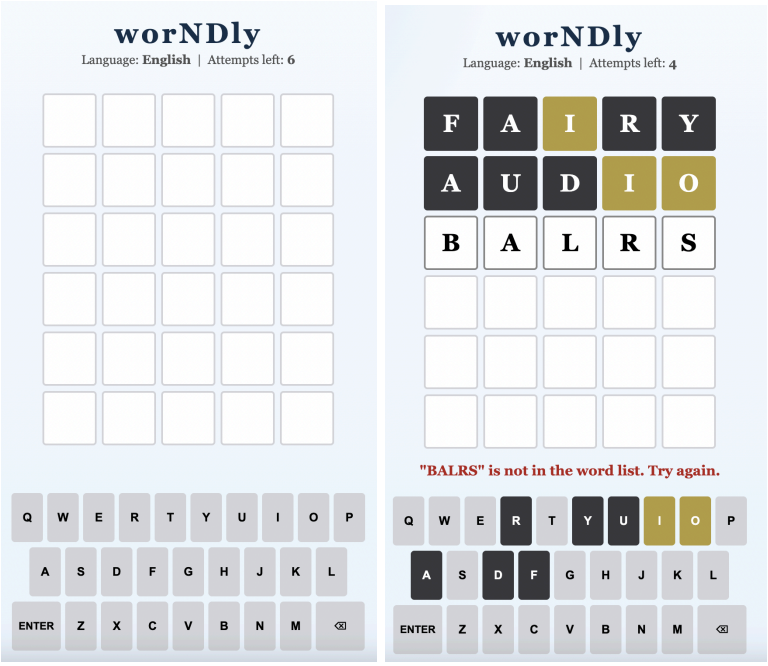

Figure 2. Screenshot of feature 2 empty grid, word checking, and invalid word

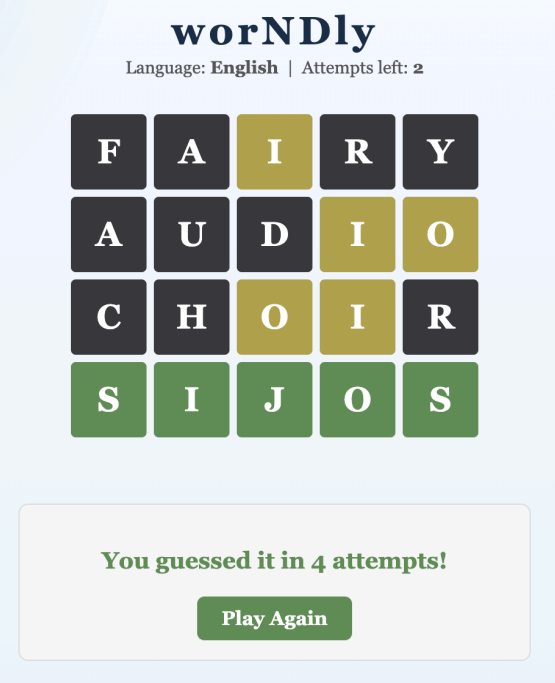

Figure 3. Screenshot of feature 2 correctly guessed wor

Figure 4. Screenshot of feature 2 incorrectly guessed word

Feature 3.1: Dashboard Table of Prior Plays

Feature 3.1 allows logged in users to access a Dashboard page containing a table with their previous game data. They can select data from this week, this month, this year, or all time. The columns displayed are date, language, result and attempts, which are accessed through the Game model in the database. Additionally, the player can directly choose to play a new game. The image below showcases the exact UI and contents of the Dashboard page.

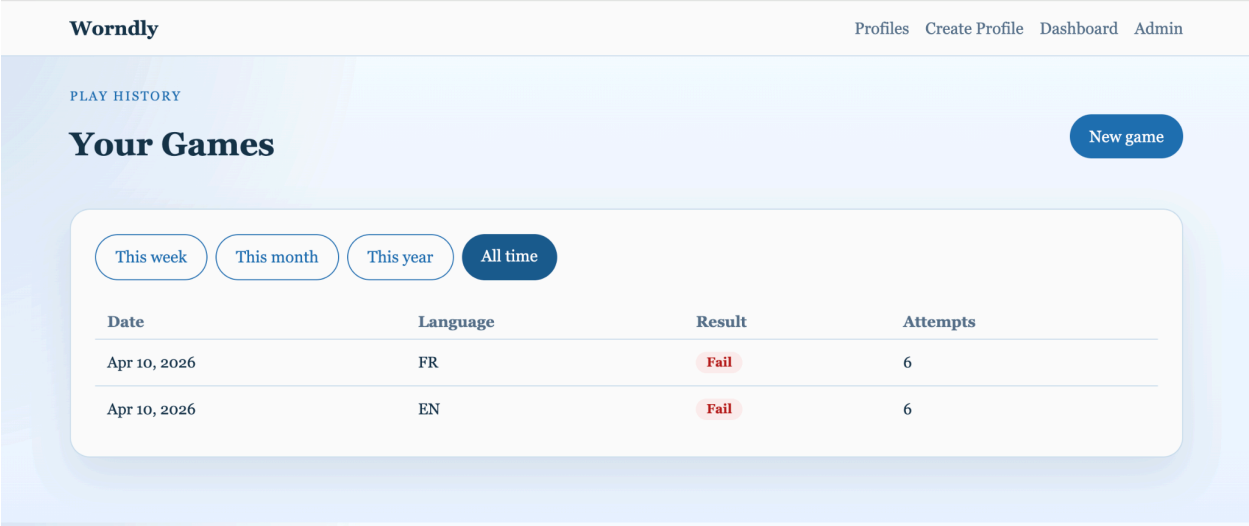

Figure 4 screenshot for feature 3.1

If the player is not logged in, clicking on the dashboard will take the player to the login page. The login feature was not fully integrated in this phase, so currently this leads to an error page, as shown in Figure 5.

Page not found (404)

Request Method: GET
Request URL: http://127.0.0.1:8000/accounts/login/?next=/dashboard/

Using the URLconf defined in `course_project.urls`, Django tried these URL patterns, in this order:

1. `admin/`
2. `[name='home']`
3. `profiles/ [name='profile_list']`
4. `profiles/new/ [name='profile_create']`
5. `profiles/<int:pk>/ [name='profile_detail']`
6. `game/ [name='game_select']`
7. `game/<int:pk>/ [name='game_play']`
8. `game/<int:pk>/guess/ [name='game_guess']`
9. `dashboard/ [name='dashboard']`

The current path, `accounts/login/`, didn't match any of these.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

Figure 5 screenshot for feature 3.1